

Обучение с подкреплением с нуля

От случайной политики до посадки на Луну

Леонтьев Михаил

ML-researcher
ЛОШ МФТИ

Содержание

1 Почему это не supervised

2 Словарь

3 Q-Learning

Мир известен

Мир неизвестен

4 DQN (Deep Q-Network)

5 RLHF

Почему это не supervised

Запускаем случайную политику

Представьте, что вам нужно научить алгоритм играть в Марио. У вас есть: приставка с этой игрой и — В С Ё.



Запускаем случайную политику

Представьте, что вам нужно научить алгоритм играть в Марио. У вас есть: приставка с этой игрой и — В С Ё.

Запускаем случайные нажатия кнопок и смотрим, что получится.



Запускаем случайную политику

Представьте, что вам нужно научить алгоритм играть в Марио. У вас есть: приставка с этой игрой и — В С Ё.

Запускаем случайные нажатия кнопок и смотрим, что получится.

Где здесь X и y ?



Три отличия от supervised learning

- 1 Нет разметки. Никто не сказал нам заранее, какое действие правильное в каждом кадре.

Три отличия от supervised learning

- 1 Нет разметки. Никто не сказал нам заранее, какое действие правильное в каждом кадре.
- 2 Выборку агент пишет себе сам. Данные — это то, что агент увидел, играя своей же (пока плохой) стратегией.

Три отличия от supervised learning

- 1 **Нет разметки.** Никто не сказал нам заранее, какое действие правильное в каждом кадре.
- 2 **Выборку агент пишет себе сам.** Данные — это то, что агент увидел, играя своей же (пока плохой) стратегией.
- 3 **Награда отложена.** Хороший ход может окупиться только через сотню шагов — как жертва фигуры в шахматах.

Три отличия от supervised learning

- 1 **Нет разметки.** Никто не сказал нам заранее, какое действие правильное в каждом кадре.
- 2 **Выборку агент пишет себе сам.** Данные — это то, что агент увидел, играя своей же (пока плохой) стратегией.
- 3 **Награда отложена.** Хороший ход может окупиться только через сотню шагов — как жертва фигуры в шахматах.

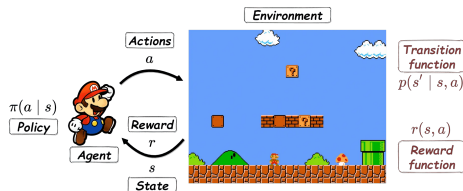
Мы не учим функцию по готовым парам (вход, ответ). Мы учим *стратегию действий*, наблюдая последствия собственных решений.

Словарь

Цикл взаимодействия

В каждый момент времени t :

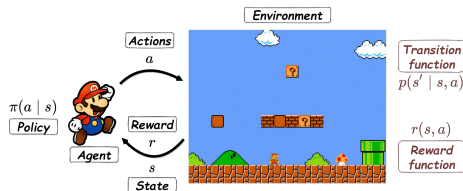
- Среда находится в состоянии (state)
 s_t



Цикл взаимодействия

В каждый момент времени t :

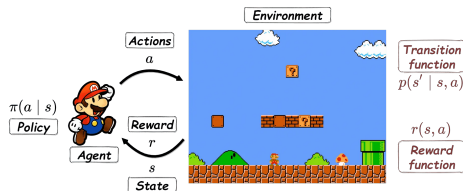
- Среда находится в состоянии (state) s_t
- Агент делает действие (action) a_t



Цикл взаимодействия

В каждый момент времени t :

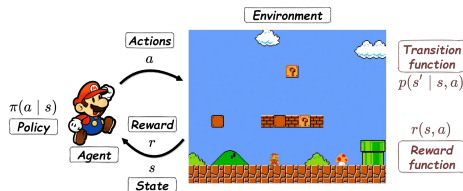
- Среда находится в состоянии (state) s_t
- Агент делает действие (action) a_t
- Агент получает награду (reward) r_t



Цикл взаимодействия

В каждый момент времени t :

- Среда находится в состоянии (state) s_t
- Агент делает действие (action) a_t
- Агент получает награду (reward) r_t
- Среда переходит в состояние s_{t+1}

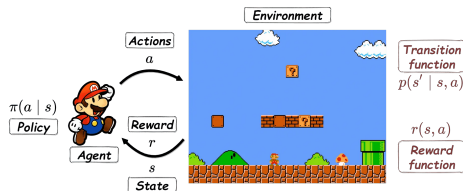


Цикл взаимодействия

В каждый момент времени t :

- Среда находится в состоянии (state) s_t
- Агент делает действие (action) a_t
- Агент получает награду (reward) r_t
- Среда переходит в состояние s_{t+1}

«Игрок» — это **policy** $\pi(a_t | s_t)$: насколько хорошо в состоянии s_t сделать действие a_t .



Примеры

Задача	Action Space	State Space	Reward
Марио			

Примеры

Задача	Action Space	State Space	Reward
Марио	Дискретные: кнопки джой- стика	Кадр экрана, позиции врагов и объектов, ско- рость персонажа	+ за прогресс и очки, - за смерть/урон
Робот- манипулятор			

Примеры

Задача	Action Space	State Space	Reward
Марио	Дискретные: кнопки джой- стика	Кадр экрана, позиции врагов и объектов, ско- рость персонажа	+ за прогресс и очки, - за смерть/урон
Робот- манипулятор	Непрерывные: моменты в мо- торах	Углы суставов, показана ния камер и датчиков	+ за успешный захват, - за время и энергию

Примеры

Задача	Action Space	State Space	Reward
Марио	Дискретные: кнопки джой- стика	Кадр экрана, позиции врагов и объектов, ско- рость персонажа	+ за прогресс и очки, - за смерть/урон
Робот- манипулятор	Непрерывные: моменты в мо- торах	Углы суставов, показа- ния камер и датчиков	+ за успешный захват, - за время и энергию

Разные пространства, разные награды — но *один и тот же* цикл взаимодействия.

Ценность

Value состояния — это «сколько награды я в среднем наберу до конца эпизода, если начну отсюда и буду следовать своей стратегии»:

Ценность

Value состояния — это «сколько награды я в среднем наберу до конца эпизода, если начну отсюда и буду следовать своей стратегии»:

$$V^{\pi}(s) = \mathbb{E}_{\pi} [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \mid s_t = s]$$

Ценность

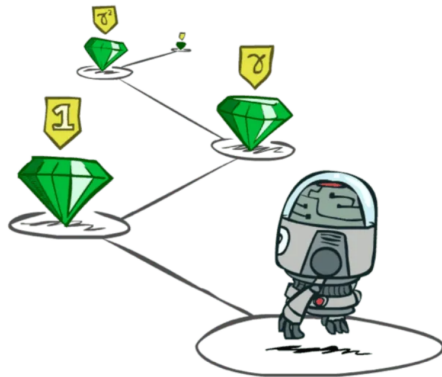
Value состояния — это «сколько награды я в среднем наберу до конца эпизода, если начну отсюда и буду следовать своей стратегии»:

$$V^{\pi}(s) = \mathbb{E}_{\pi} [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \mid s_t = s]$$

Это то же самое, что оценка позиции в шахматах: не «сколько фигур на доске прямо сейчас», а «насколько вероятно из этой позиции выиграть партию».

Дисконт γ

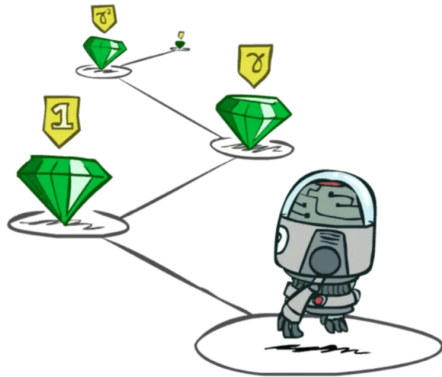
Коэффициент $\gamma \in [0, 1]$ — насколько далеко вперёд агент готов планировать.



Дисконт γ

Коэффициент $\gamma \in [0, 1]$ — насколько далеко вперёд агент готов планировать.

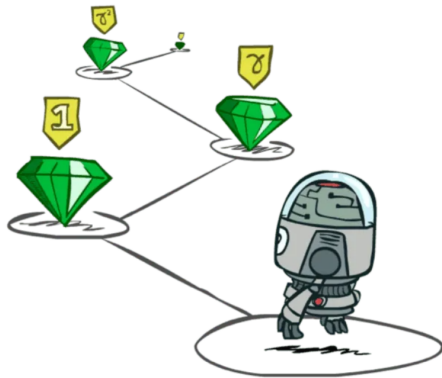
- $\gamma \rightarrow 0$: важен только следующий шаг — «близорукий» агент.



Дисконт γ

Коэффициент $\gamma \in [0, 1]$ — насколько далеко вперёд агент готов планировать.

- $\gamma \rightarrow 0$: важен только следующий шаг — «близорукий» агент.
- $\gamma \rightarrow 1$: одинаково важны и ближайшая, и далёкая награда — «дальновидный» агент.

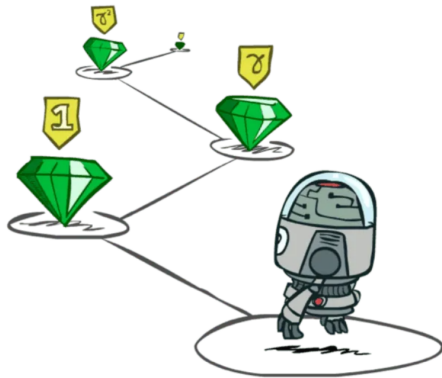


Дисконт γ

Коэффициент $\gamma \in [0, 1]$ — насколько далеко вперёд агент готов планировать.

- $\gamma \rightarrow 0$: важен только следующий шаг — «близорукий» агент.
- $\gamma \rightarrow 1$: одинаково важны и ближайшая, и далёкая награда — «дальновидный» агент.

γ — это горизонт планирования, зашитый в формулу.



Мир известен

Cliff Walking

Карта-обрыв: старт слева внизу,
цель справа внизу, вдоль всего
нижнего края — пропасть.



Cliff Walking



Карта-обрыв: старт слева внизу, цель справа внизу, вдоль всего нижнего края — пропасть.

Вдоль края — короткий, но опасный маршрут.

Cliff Walking



Карта-обрыв: старт слева внизу, цель справа внизу, вдоль всего нижнего края — пропасть.

Вдоль края — короткий, но опасный маршрут.

Через верх — длиннее, но безопасный.

Cliff Walking



Карта-обрыв: старт слева внизу, цель справа внизу, вдоль всего нижнего края — пропасть.

Вдоль края — короткий, но опасный маршрут.

Через верх — длиннее, но безопасный.

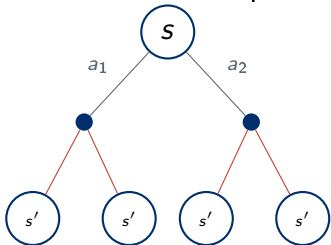
Какой из них окажется оптимальным?

Уравнение Беллмана

Если мир нам известен (мы знаем $P(s' | s, a)$ и $r(s, a)$ для *любых* s, a),
ценность состояния раскладывается на «шаг + остальное»:

Уравнение Беллмана

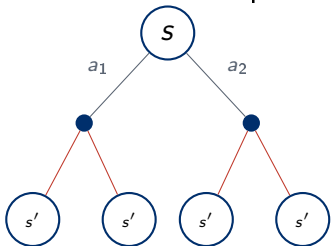
Если мир нам известен (мы знаем $P(s' | s, a)$ и $r(s, a)$ для *любых* s, a), ценность состояния раскладывается на «шаг + остальное»:



$$V(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

Уравнение Беллмана

Если мир нам известен (мы знаем $P(s' | s, a)$ и $r(s, a)$ для любых s, a), ценность состояния раскладывается на «шаг + остальное»:



$$V(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

Словами: *ценность состояния = лучшая награда за один шаг + продисконтированная ценность того, куда мы попадём.*

Value Iteration

- 1 Инициализируем $V(s) = 0$ для всех s .

Value Iteration

- 1 Инициализируем $V(s) = 0$ для всех s .
- 2 На каждой итерации обновляем *все* состояния разом:

$$V(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

Value Iteration

- 1 Инициализируем $V(s) = 0$ для всех s .
- 2 На каждой итерации обновляем *все* состояния разом:

$$V(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

- 3 Повторяем, пока V не перестанет меняться.

Value Iteration

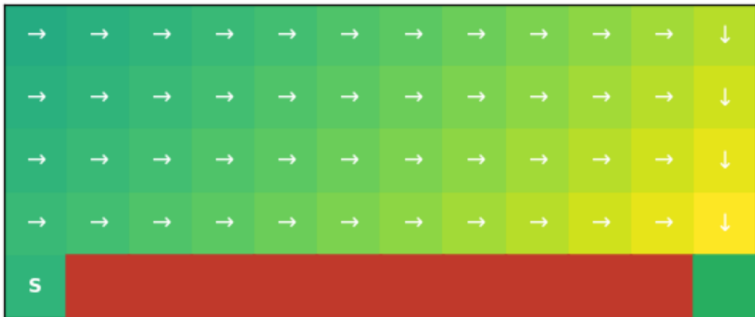
- 1 Инициализируем $V(s) = 0$ для всех s .
- 2 На каждой итерации обновляем *все* состояния разом:

$$V(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

- 3 Повторяем, пока V не перестанет меняться.
- 4 Политика: $\pi(s) = \arg \max_a \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$.

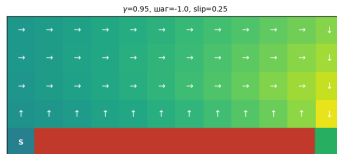
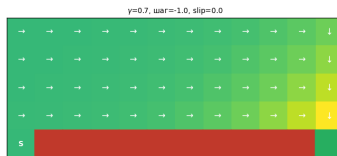
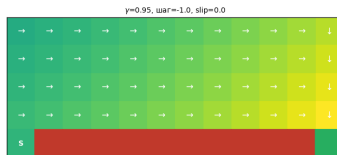
Демо №1. Волна ценности

Value Iteration, итерация 40



Value Iteration: как оценка растекается от цели к старту по мере итераций

Демо №2. Три ручки



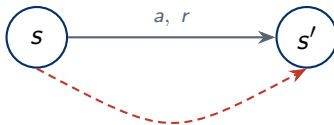
Мир неизвестен

Q-learning: идея

Раз модели мира нет — будем действовать и *подвигать* оценку в сторону того, что реально увидели.

Q-learning: идея

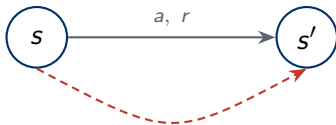
Раз модели мира нет — будем действовать и *подвигать* оценку в сторону того, что реально увидели.



цель: $r + \gamma \max_{a'} Q(s', a')$

Q-learning: идея

Раз модели мира нет — будем действовать и *подвигать* оценку в сторону того, что реально увидели.



цель: $r + \gamma \max_{a'} Q(s', a')$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{увидели}} - \underbrace{Q(s, a)}_{\text{думали}} \right]$$

Exploration vs. Exploitation



Exploration vs. Exploitation



Какая стратегия здесь оптимальна?

ε -greedy

Чтобы не заиклиться на первом найденном решении, действуем так:

- с вероятностью $1 - \varepsilon$ — жадно, $a = \arg \max_a Q(s, a)$;

ϵ -greedy

Чтобы не заиклиться на первом найденном решении, действуем так:

- с вероятностью $1 - \epsilon$ — жадно, $a = \arg \max_a Q(s, a)$;
- с вероятностью ϵ — случайно, чтобы попробовать что-то новое.

ϵ -greedy

Чтобы не заиклиться на первом найденном решении, действуем так:

- с вероятностью $1 - \epsilon$ — жадно, $a = \arg \max_a Q(s, a)$;
- с вероятностью ϵ — случайно, чтобы попробовать что-то новое.

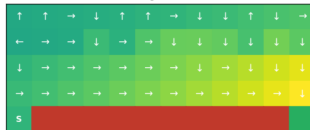
Компромисс: слишком мало исследования — рискуем не найти хорошую стратегию; слишком много — слишком часто действуем случайно.

Демо №3. Q-learning рядом с эталоном

Мир известен: эталон



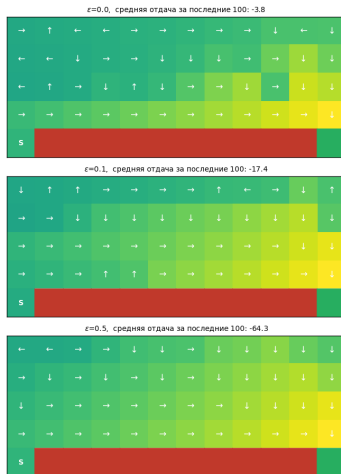
Q-learning, эпизод 395



Где агент вообще был (log-шкала)



Демо №4. Что бывает без исследования



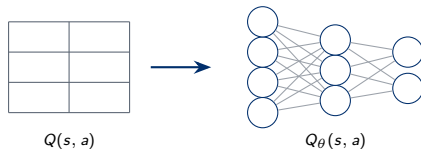
DQN

От таблицы к сети

Таблицу $Q(s, a)$ нельзя выписать — значит, нужно её *приблизить*.

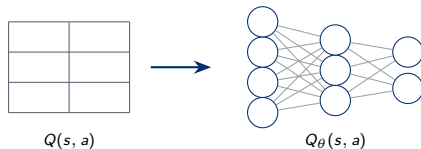
От таблицы к сети

Таблицу $Q(s, a)$ нельзя выписать — значит, нужно её *приблизить*.



От таблицы к сети

Таблицу $Q(s, a)$ нельзя выписать — значит, нужно её *приблизить*.



Обучаем той же формулой, но двигаем не ячейку таблицы, а веса сети:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \left(Q_\theta(s, a) - [r + \gamma \max_{a'} Q_\theta(s', a')] \right)^2$$

Три поломки, три заплатки

Поломка	Заплата
Соседние шаги коррелированы	Replay buffer : учимся на случайном перемешанном наборе прошлых переходов

Три поломки, три заплатки

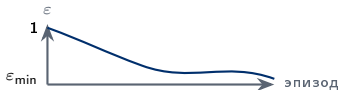
Поломка	Заплата
Соседние шаги коррелированы	Replay buffer : учимся на случайном перемешанном наборе прошлых переходов
Таргет зависит от θ	Замороженная копия $Q_{\theta-}$: цель считаем по старой, редко обновляемой копии

Три поломки, три заплатки

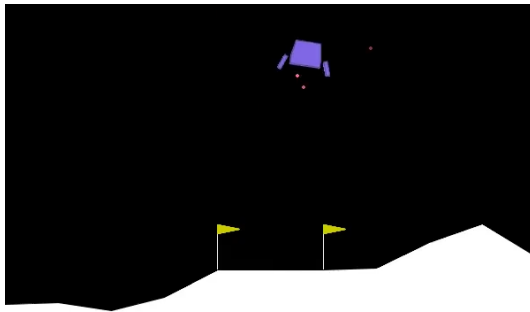
Поломка	Заплата
Соседние шаги коррелированы	Replay buffer : учимся на случайном перемешанном наборе прошлых переходов
Таргет зависит от θ	Замороженная копия $Q_{\theta-}$: цель считаем по старой, редко обновляемой копии
Жадная политика с начала — сеть не успевает исследовать	ε-расписание : ε убывает от 1 до малого значения

Три поломки, три заплатки

Поломка	Заплата
Соседние шаги коррелированы	Replay buffer : учимся на случайном перемешанном наборе прошлых переходов
Таргет зависит от θ	Замороженная копия $Q_{\theta-}$: цель считаем по старой, редко обновляемой копии
Жадная политика с начала — сеть не успевает исследовать	ϵ-расписание : ϵ убывает от 1 до малого значения

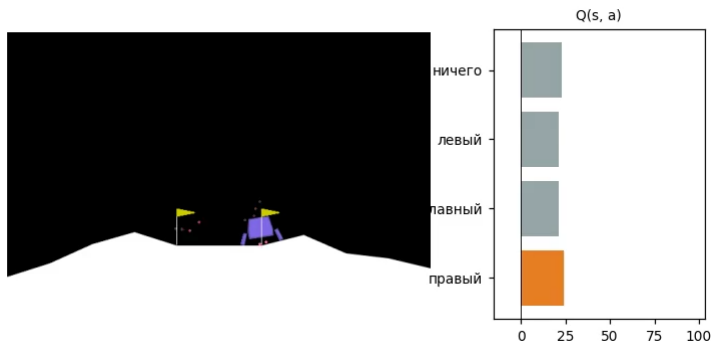


Демо №5. LunarLander: три ролика



Агент садится между флажками — задача с непрерывным состоянием и дискретными действиями

Демо №6. Q-значения поверх видео

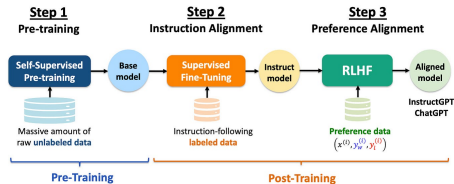


$Q(s, a)$ для каждого действия в текущем кадре: видно, какое действие сеть считает лучшим

RLHF

Мостик к RLHF

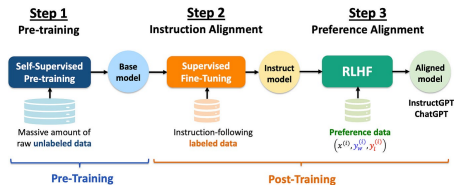
У LLM то же самое: reward — это не человек с кнопкой, а обученная *модель предпочтений*.



Мостик к RLHF

У LLM то же самое: reward — это не человек с кнопкой, а обученная *модель предпочтений*.

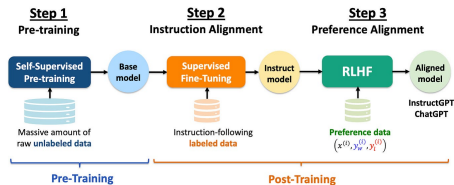
- state — промпт + написанная часть ответа;



Мостик к RLHF

У LLM то же самое: reward — это не человек с кнопкой, а обученная *модель предпочтений*.

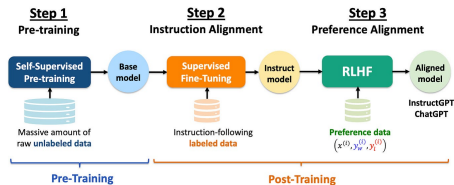
- state — промпт + написанная часть ответа;
- action — очередной токен;



Мостик к RLHF

У LLM то же самое: reward — это не человек с кнопкой, а обученная *модель предпочтений*.

- state — промпт + написанная часть ответа;
- action — очередной токен;
- reward — оценка reward-модели, обученной на парах «какой ответ лучше».

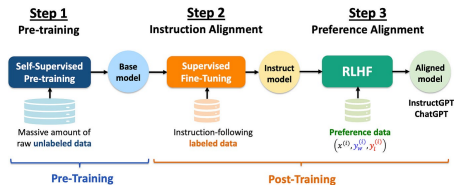


Мостик к RLHF

У LLM то же самое: reward — это не человек с кнопкой, а обученная *модель предпочтений*.

- state — промпт + написанная часть ответа;
- action — очередной токен;
- reward — оценка reward-модели, обученной на парах «какой ответ лучше».

Тот же цикл, то же уравнение Беллмана — просто состояние и награда устроены иначе.



Куда дальше

Q-learning и DQN — это *value-based* методы: учим ценность, действие выбираем через $\arg \max$. Хорошо для дискретных действий (4 двигателя «вкл/выкл»).

Куда дальше

Q-learning и DQN — это *value-based* методы: учим ценность, действие выбираем через $\arg \max$. Хорошо для дискретных действий (4 двигателя «вкл/выкл»).

$\arg \max_a Q(s, a)$ по непрерывному a решать дорого. Нужен другой класс методов — *policy gradient*: учим стратегию напрямую (REINFORCE → Actor-Critic → PPO).

Спасибо за внимание!

Леонтьев Михаил • ЛОШ МФТИ

Telegram



https://t.me/michlea_tg

Материалы



<https://github.com/Leamich/HSE-RL-slides>